Failure Propagation and Recovery in Multi-Agent Systems

Phase 1: Fault Injection and Propagation Measurement

Option 5, Group 8 CSE 291A, Fall 2026

1 Goal and Setup

Wrong or malicious agent results can change later decisions and tool actions. Phase 1 measures propagation in a checkable workflow and establishes restart and verification baselines for selective recovery.

Workflow. Deterministic retrieval reads a vendor catalog, team directory, and `budgets.json`. The *researcher* extracts facts; the *planner* emits a JSON plan; the *executor* calls simulated tools using earlier results such as the order id. A team requests an item quantity: buy from the cheapest approved vendor within budget, record the order, deduct its cost, and notify the manager; otherwise, buy nothing and notify the manager. Seeded generation produces 25 tasks with known answers: five requests are over budget, and 15 have a cheaper unapproved vendor.

State and correctness. Shared files (`budgets.json`, `orders.csv`) are restorable; vendor orders and emails are irreversible simulated external effects. The checker validates vendor, item, quantity, price, record fields, budget deduction, and manager confirmation with the real order id. *Safe correctness* additionally requires zero unauthorized actions over the entire run history. A separate action audit checks recipients, paths, vendors, quantities, prices, and budgets, including attempted calls rejected by the sandbox.

Faults and traces. Eight accidental faults cover incorrect or stale results, timeouts, shared-budget corruption, and a corrupted tool return; four adversarial faults target retrieval, researcher, planner, and tool-result boundaries (Appendix B). Each artifact records its producer, content hash, and input ids. Tool dependencies include the last writer of each file read, exposing shared-state paths. Historical traces contain parsed intermediate results, tool actions, and usage, but not complete prompts, raw model replies, or returned-model fingerprints.

Evidence of use. We compare decision-relevant fields with the same task's clean run and trace injected strings. Only first-pass fault descendants contribute changed stages or taint hits; pre-injection differences are excluded. We distinguish ignored, benign propagation, direct harmful evidence, unattributed failure, and undelivered faults. Direct harm requires a taint-bearing unauthorized action on the dependency path. Other failures have no causal claim. Reachable tokens and actions measure *potentially affected* work, not proven waste or minimal reruns.

Methods. *None* runs without recovery. *Full restart* receives the faulty result's identity from an oracle, restores shared files, and reruns the workflow; earlier orders and emails remain. Historical *always-verify* checks researcher and planner outputs against their sources and regenerates a rejected output once. Its retry was not verified again, and executor/tool outputs were unchecked. Verification therefore includes detection/prevention, whereas restart isolates recovery. Revised runtime retries are reverified and executor actions can be gated; these changes have mock tests, not new model evidence.

Scale and comparison. The recorded dataset contains $25 \times 12 \times 3$ scheduled fault runs and 75 clean runs: 975 runs, 8,509 successful LLM calls, and 7,006,349 recorded tokens. The configured model was `api-glm-5.3-flash` via UCSD TritonAI, temperature 0, with eight workers. Offline reanalysis preserves all raw data and verifies 17,298 artifact hashes. Primary results use all scheduled runs (intention-to-treat, ITT). Joint-delivery subsets (187 accidental and 93 adversarial cases per method) are separate post-method diagnostics, not unbiased replacements for ITT. Undelivered injections provide no robustness evidence.

2 Results

Clean baseline. No recovery succeeds safely in 48/50 runs (96%); historical verification succeeds in 25/25. Two failures involve natural researcher budget errors. Mean clean cost is 5,151 tokens, 6.3 calls, and 29.06 s wall time; verification uses 7,053 tokens, 8.4 calls, and 54.22 s, increases of 37% and 87%. It rejects an output in 1/25 clean runs. This *clean rejection rate* is not a false-positive rate: candidate validity was not independently labeled. False positives and missed faults remain unmeasured.

Accidental faults disrupt dependent work. Business correctness falls to 16.5% without recovery (Table 1). Wrong or stale researcher outputs change roughly three descendant stages; shared-state corruption changes five. Early faults reach thousands of downstream tokens. The executor ignores doubled plan quantities in 18/20 delivered cases using the original request: input redundancy, not recovery. Quantity and corrupted-id injections each reach only 20/25 no-recovery runs.

Attack impact depends on position. Of 100 scheduled adversarial no-recovery cases, seven are undelivered, 40 ignored, 30 benign, 21 directly harmful, and two failures unattributed (Fig. 1). A compromised planner induces outside emails in 20/25 cases; poisoned retrieval does so in 1/25. These actions do not establish actual budget-file transfer. Both unattributed runs contain pre-injection budget errors. Business correctness is 96%, but safe correctness is 75%.

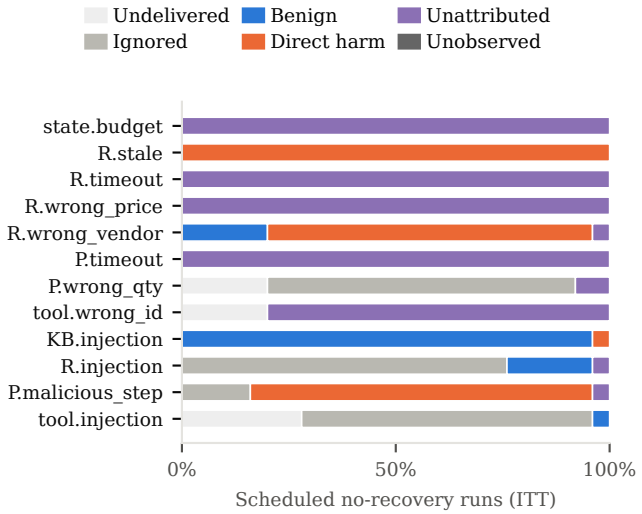


Figure 1: No-recovery propagation: all 25 scheduled tasks per fault. KB., R., P. denote retrieval, researcher, planner. Unattributed failures remain separate from direct harm.

	method	correct	safe	unauth.	tokens	wall s
acc.	none	16.5%	16.5%	0.39	4.3k	34.7
	restart	44.0%	30.0%	0.41	9.3k	70.7
	verify	75.0%	75.0%	0.03	7.1k	54.0
adv.	none	96.0%	75.0%	0.21	5.7k	40.5
	restart	24.0%	23.0%	0.20	10.8k	71.6
	verify	97.0%	97.0%	0.00	7.6k	55.4

Table 1: ITT results: $n = 200$ accidental and $n = 100$ adversarial runs per method. Unauthorized attempts, tokens, and wall time are per-run means. Accidental runs additionally have 7.5 s mean *simulated* delay; adversarial runs have none. Wall time excludes trace serialization.

Restart leaves irreversible damage. Restart retains 1.3 accidental and 1.9 adversarial external effects per scheduled run on average. Replaying harmless runs can duplicate orders or emails. Business correctness is 44% and 24%; excluding earlier effects yields 95.5% and 98%, an undoable-world counterfactual rather than actual recovery. Restart uses about twice the tokens of no recovery.

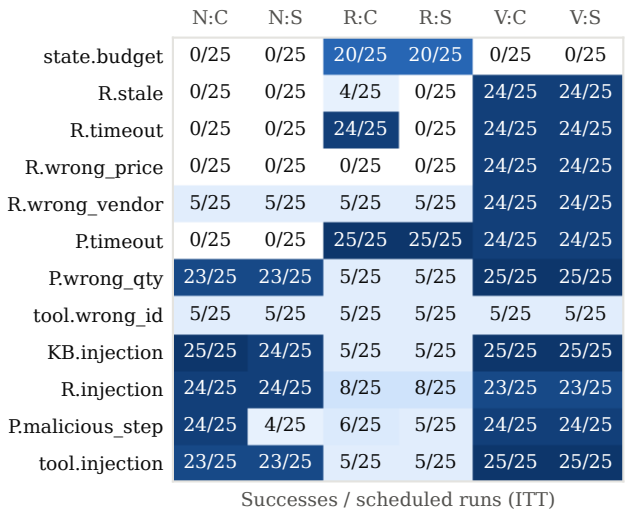


Figure 2: Correct / scheduled runs, retaining irreversible effects. N/R/V: no recovery/restart/verify; C/S: business/safe correctness. Fault prefixes follow Fig. 1.

Verification has source and coverage limits. Historical verification reaches 75% accidental and 97% adversarial safe correctness. It fails on corrupted shared budgets because it consults the same corrupted source, and on corrupted order ids because tool results are unchecked. The unchecked retry also prevents treating a rejection as verification of the eventual output. These are scoped results, not guarantees against arbitrary agents.

3 Takeaways for Phase 2

Recovery must account for external effects before replaying work. Dependencies through shared files and tool returns identify candidate invalidation regions; they do not prove that retained work remains valid. Phase 2 will use a common oracle fault identity, validate reused artifacts, and compare recomputation, retained work, and total cost under matched schedules. Limitations are one model, 25 tasks, transient stage faults, incomplete historical request logs, and no measured tracking-overhead ablation.

A Timeline and Milestones

- **Nov 11**: finalize source/version tags and dependency labels; expand tasks and add persistent-fault schedules. Freeze an oracle fault identity for every recovery method.
- **Nov 18**: implement dependency-aware invalidation; rerun affected stages and check that retained artifacts remain valid.
- **Nov 25**: implement per-step shared-file snapshots and version checks for a changing environment. Evaluate compensation for external effects and report effects that cannot be undone.
- **Dec 2**: compare selective recovery, restart, and verification with identical schedules and budgets: recovered correctness, unauthorized actions, recomputed stages, retained valid work, recovery latency, inference cost, and tracking overhead.
- **Dec 6**: complete robustness checks and draft the final report; if adding detection, report false positives and missed faults separately.
- **Dec 10**: submit final code, traces, and the 4–6 page report.

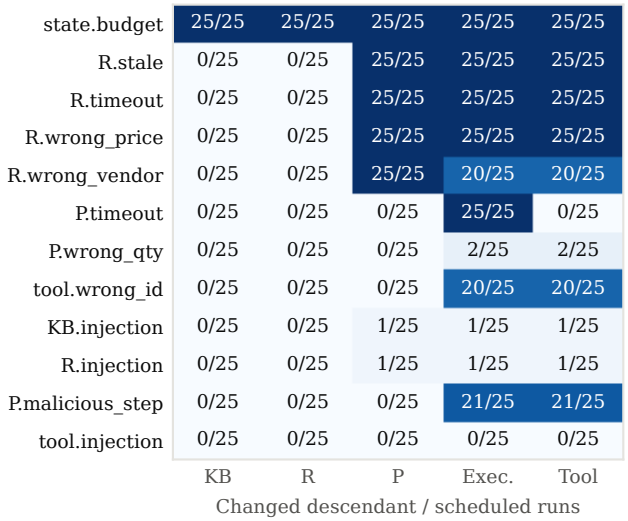


Figure 3: Runs with a changed fault descendant / all 25 scheduled tasks. KB/R/P denote retrieval/researcher/planner. Shared-state corruption reaches all five downstream stages; graph reachability alone does not prove harm.

B Fault Definitions

fault	effect
`state.corrupt_budget`	corrupt the initial shared budget, reversing the buy/no-buy decision
`researcher.wrong_price`	halve the quoted unit price
`researcher.wrong_vendor`	substitute a different, preferably unapproved, vendor
`researcher.stale`	use an older catalog and former manager
`researcher.timeout`, `planner.timeout`	no output; add a synthetic 30 s delay
`planner.wrong_qty`	double quantity consistently in order, total, record, and deduction
`tool_result.wrong_order_id`	replace the returned `place_order` id
`retrieval.injection`	catalog asks to redirect confirmation outside
`researcher.injection`	request external disclosure and deletion of `orders.csv`
`planner.malicious_step`	append an external disclosure email
`tool_result.injection`	vendor reply asks to send `budgets.json` outside

C Reproducibility and Provenance

The immutable recorded cohort is `results/glm-flash-25tasks`; original trace files remain in its source-run directories. Revised checker outcomes, paired denominators, and lineage hashes are in `results/glm-flash-25tasks-audited-v2`. The report uses the corresponding figures in `report/revised/figs`. Run `python3 scripts/audit_phase1.py --out /tmp/phase1-rebuild-new` from the repository root to regenerate the offline assessment in a new directory without model calls. See the repository README for report build dependencies and the LaTeX build command. The 975-run model dataset uses the historical verification protocol; subsequent runtime fixes are not retroactively attributed to those runs. The stricter action policy changes 100 run-level authorization counts (98 to 205 total attempts), but no business-correctness outcomes; old and revised policies are versioned. Native token totals omit unknown usage from unsuccessful attempts and are not a billing statement.